RESUME SCREENER

AI Resume Screener is a fully serverless full-stack web application that allows recruiters to upload PDF resumes and instantly receive an AI-powered screening score. The system automatically extracts text from each resume, matches it against 20 technical keywords with weighted scoring, assigns a rank (Excellent / Good / Average / Below Average), and displays all results in a ranked dashboard. The entire backend is built in Python and runs on AWS Lambda — no servers to manage, infinitely scalable.

Component	**Technology**	**Purpose**
Frontend	React 18 + Vite	User interface - upload + dashboard
Hosting	AWS Amplify	CI/CD deployment from GitHub
API Layer	AWS API Gateway (HTTP)	Routes HTTP requests to Lambda
Backend 1	Lambda-get_presigned_url.py (Python)	Generates secure S3 upload URL
Backend 2	Lambda- process_resume.py (Python)	Extracts text, scores, ranks resume
Backend 3	Lambda - get_resumes.py (Python)	Fetches all resumes sorted by score
Database	AWS DynamoDB	Stores resume data — NoSQL, serverless
Strorage	AWS S3	Stores uploaded PDF files

Working of project :

1. **User uploads PDF :** React frontend calls POST /upload-url → get_presigned_url Lambda returns a secure S3 presigned URL valid for 5 minutes
2. **Direct S3 upload :** Browser uploads the PDF directly to S3 using the presigned URL — file never passes through Lambda (avoids 6MB limit)
3. **Auto trigger :** S3 PUT event automatically triggers process_resume Lambda — no manual call needed
4. **Text Extraction : L**ambda downloads the PDF from S3 into memory and extracts all text using PyPDF2
5. **Keyword scoring :** Python scores the text against 20 technical keywords with weighted points (python=10, aws=10, docker=7 etc.)
6. **Rank assignment :** Score >= 80 = Excellent, >= 60 = Good, >= 40 = Average, else = Below Average
7. **DynamoDB save :** Result saved as one record: resumeId, candidateName, email, score, rank, matchedKeywords, uploadedAt
8. **Dashboard fetch :** React calls GET /resumes → get_resumes Lambda scans DynamoDB, sorts by score desc, returns JSON
9. **Ranked Display :** Frontend renders ranked table with score, colour-coded rank badge, matched keywords, date.

Keyword Scoring System :

Keyword	Points	Keyword	Points	Keyword	Points
Python	10	Java	8	Javascript	8
Aws	10	Machine learning	9	Docker	7
Django	7	Flask	7	React	7
Sql	6	Linux	6	Mongodb	6
Rest	6	Api	6	Git	5
Agile	5	Database	5	html	4
Css	4				

Rank	Score range	Meaning
Excellent	80 and above	Strong candidate - most keywords matched
Good	60 to 79	Solid candidate - good keyword coverage
Average	40 to 59	Some relevant skills - may need review
Below average	Below 40	Few matching keywords - likely not a fit

```
Resume-screener/
→ frontend/                    React app (Vite)
→ src/
        App.jsx        Main dashboard page
        main.jsx       Entry point
→ components/
            UploadSection.jsx          File upload + S3 upload logic
            StatsBar.jsx               4 summary stat cards
            ResumeTable.jsx        Ranked results table
→ .env                         API Gateway URL
→ package.json
→ backend/
→ lambda_functions/
        get_presigned_url.py       Lambda 1 — S3 presigned URL
        process_resume.py          Lambda 2 — PDF scoring engine
        get_resumes.py             Lambda 3 — fetch + sort results
```

Local Setup :

- Node.js 20+ and Python 3.11+ must be installed
- AWS CLI configured with ap-south-1 region
- AWS account with S3, DynamoDB, Lambda, API Gateway access

Clone and run :

```
git clone https://github.com/YOURUSERNAME/resume-screener.git

cd resume-screener/frontend

npm install

npm install axios

# create .env with: VITE_API_URL=your_api_gateway_url

npm run dev
```

AWS Deployment :

1. S3 bucket created with CORS configured for direct browser uploads
2. DynamoDB table resumes with partition key resumeId (String)
3. Lambda function process-resume triggered by S3 PUT events on .pdf files
4. Lambda function get-resumes triggered by GET /resumes API Gateway route
5. Lambda function get-presigned-url triggered by POST /upload-url route
6. API Gateway HTTP API with CORS enabled, deployed to $default stage
7. Frontend deployed on AWS Amplify with VITE_API_URL environment variable

Skill Demonstrated :

React 18	Component-based UI, useState, useEffect, props, axios
Python 3.11	Lambda functions, boto3 SDK, PyPDF2, regex, JSON handling
AWS Lambda	Serverless compute, event-driven triggers, IAM roles
AWS DynamoDB	NoSQL database, scan, put_item, sort operations
AWS S3	Object storage, presigned URLs, event notifications
AWS API Gateway	HTTP API, route configuration, CORS, Lambda integration
AWS Amplify	CI/CD pipeline, GitHub integration, env variables
Git + Github	Version control, branching, push, remote repository